

Plano de Teste — Benefit Packages API

Cobertura funcional para POST, GET Collection, GET Detail e PATCH do endpoint de Benefit Packages.

Preparação

- Configurar {{baseUrl}} e {{token}} no Postman.
- Headers: Content-Type: application/json e authorization2: Bearer {{token}}.
- Executar primeiro um POST válido e salvar o benefit_package_id como {{benefitPackageId}}.
- Após PATCHs de sucesso, executar GET Detail para validar o estado persistido.

POST — Sucesso

ID	Cenário	Resultado esperado
POST-01	Criação válida	201; ID retornado; Draft; Term 12; 2 ContractLineItems; Core UnitPrice 0; Optional UnitPrice
POST-02	Sem optional_benefits	201; somente Core criado.
POST-03	optional_benefits vazio	201.

POST — Required Fields

ID	Cenário	Resultado esperado
POST-04	Sem product_reference	400
POST-05	Sem active_start_at	400
POST-06	Sem active_end_at	400
POST-07	Sem amount_of_policies	400
POST-08	Sem core_benefits	400
POST-09	Sem legal_consents	400
POST-10	Core sem benefit_id	400
POST-11	Optional sem benefit_id	400
POST-12	Optional sem name	400

POST — Validações de negócio

ID	Cenário	Resultado esperado
POST-13	amount_of_policies = 0	400
POST-14	amount_of_policies negativo	400
POST-15	End date igual Start date	400
POST-16	End date anterior ao Start date	400
POST-17	Data inválida	400
POST-18	Policy range sem start	400
POST-19	Policy range sem end	400

POST — Dados / Segurança

ID	Cenário	Resultado esperado
POST-20	Product inexistente	400

ID	Cenário	Resultado esperado
POST-21	Benefit inexistente	400
POST-22	Benefit sem Product2	400
POST-23	Benefit sem PricebookEntry ativo	400
POST-24	Sem authorization	401
POST-25	Token inválido	401
POST-26	Resource path inválido	404
POST-27	JSON inválido	400

GET — Collection

ID	Cenário	Resultado esperado
GET-01	Listar packages	200; array; campos principais; Content-Range.
GET-02	Paginação válida	200
GET-03	start_index = 2000	Resposta conforme quantidade disponível.
GET-04	start_index > 2000	400; mensagem de limite.
GET-05	Sem authorization	401

GET — Detail

ID	Cenário	Resultado esperado
GET-DETAIL-01	Package existente	200; validar campos, Core, Optional e Legal Consent.
GET-DETAIL-02	Package inexistente	404
GET-DETAIL-03	Sem authorization	401
GET-DETAIL-04	Resource path inválido	404

PATCH — Sucesso

ID	Cenário	Resultado esperado
PATCH-01	Atualização completa	204 sem body; depois GET Detail confirma datas, quantidade, policy range e benefícios.
PATCH-02	Remover optional_benefits	204; GET Detail confirma optional_benefits vazio.
PATCH-03	Remover affected_policy_range	204; GET Detail confirma campos nulos.

PATCH — Segurança / Resource

ID	Cenário	Resultado esperado
PATCH-04	Sem authorization	401
PATCH-05	Token inválido	401
PATCH-06	Package inexistente	404
PATCH-07	Resource path inválido	404

PATCH — Body / Required Fields

ID	Cenário	Resultado esperado
PATCH-08	Body vazio	400
PATCH-09	JSON inválido	400
PATCH-10	Sem product_reference	400
PATCH-11	Sem active_start_at	400
PATCH-12	Sem active_end_at	400
PATCH-13	Sem amount_of_policies	400
PATCH-14	Sem core_benefits	400
PATCH-15	Sem legal_consent	400

PATCH — Validações de negócio

ID	Cenário	Resultado esperado
PATCH-16	amount_of_policies = 0	400
PATCH-17	amount_of_policies negativo	400
PATCH-18	Start = End	400
PATCH-19	End < Start	400
PATCH-20	Data inválida	400
PATCH-21	Policy range sem start	400
PATCH-22	Policy range sem end	400

PATCH — Dados inexistentes

ID	Cenário	Resultado esperado
PATCH-23	Product inexistente	400
PATCH-24	Benefit inexistente	400
PATCH-25	Benefit sem PricebookEntry	400

PATCH — Regra de status

ID	Cenário	Resultado esperado
PATCH-26	Package com status Active	400; atualização não permitida.
PATCH-27	Package em Draft	204; atualização permitida.

Ordem recomendada

1. POST-01 → salvar {{benefitPackageId}}.
2. GET-01 e GET-DETAIL-01.
3. PATCH-01 → GET-DETAIL-01 para validar persistência.
4. PATCH-02 → GET-DETAIL-01 para confirmar substituição das linhas.
5. PATCH-03 → GET-DETAIL-01 para confirmar remoção do policy range.
6. Executar os cenários negativos de POST.
7. Executar segurança e paths inválidos.
8. Executar validações negativas de PATCH.

9. Alterar o package para Active e executar PATCH-26.

10. Retornar o package para Draft antes dos testes que dependem de atualização.

Critério principal de validação

Para POST e PATCH de sucesso, não validar somente o HTTP status. Executar GET Detail depois da operação e confirmar o estado persistido, verificando ContractLineItems, Quantity, UnitPrice, benefícios, datas, policy range, legal consent e Term.